Classification_Knn_NaiveBayes

November 22, 2025

[1]:
```
!pip install scikit-plot
!pip install scipy==1.11.4
```

```
Requirement already satisfied: scikit-plot in /usr/local/lib/python3.12/dist-
packages (0.3.7)
Requirement already satisfied: matplotlib>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-plot) (3.10.0)
Requirement already satisfied: scikit-learn>=0.18 in
/usr/local/lib/python3.12/dist-packages (from scikit-plot) (1.6.1)
Requirement already satisfied: scipy>=0.9 in /usr/local/lib/python3.12/dist-
packages (from scikit-plot) (1.11.4)
Requirement already satisfied: joblib>=0.10 in /usr/local/lib/python3.12/dist-
packages (from scikit-plot) (1.5.2)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot)
(1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib>=1.4.0->scikit-plot) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot)
(4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot)
(1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-
packages (from matplotlib>=1.4.0->scikit-plot) (1.26.4)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot)
(25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
packages (from matplotlib>=1.4.0->scikit-plot) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot)
(3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot)
(2.9.0.post0)
Requirement already satisfied: threadpoolctl>=3.1.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->scikit-plot)
(3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib>=1.4.0->scikit-plot) (1.17.0)
Requirement already satisfied: scipy==1.11.4 in /usr/local/lib/python3.12/dist-
packages (1.11.4)
Requirement already satisfied: numpy<1.28.0,>=1.21.6 in
/usr/local/lib/python3.12/dist-packages (from scipy==1.11.4) (1.26.4)
```

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

[3]:
```
import scipy
scipy.__version__
```

[3]:
```
'1.11.4'
```

[4]:
```
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
)
from scikitplot.metrics import plot_roc
from scikitplot.metrics import plot_precision_recall
```

[5]:
```
from sklearn.datasets import load_iris

frame = load_iris(as_frame=True)
df_orig = frame['data']
X = df_orig.values
y = np.array(frame['target'])
```

[6]:
```
df_orig.head()
```

[6]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

[7]:
```
np.unique(y, return_counts=True)
```

[7]:
```
(array([0, 1, 2]), array([50, 50, 50]))
```

[8]:
```
df_orig.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
dtypes: float64(4)
memory usage: 4.8 KB
```

0.1 Partitioning

[9]:
```
from sklearn.model_selection import train_test_split
```

[10]:
```
random_state = 0
```

[11]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=random_state
)
```

[12]:
```
# without stratify
print(np.unique(y, return_counts=True)[1] / len(y))
print(np.unique(y_train, return_counts=True)[1] / len(y_train))
print(np.unique(y_test, return_counts=True)[1] / len(y_test))
```

```
[0.33333333 0.33333333 0.33333333]
[0.37777778 0.3        0.32222222]
[0.26666667 0.38333333 0.35      ]
```

[13]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, stratify=y, random_state=random_state
)
```

[14]:
```
# with stratify
print(np.unique(y, return_counts=True)[1] / len(y))
print(np.unique(y_train, return_counts=True)[1] / len(y_train))
print(np.unique(y_test, return_counts=True)[1] / len(y_test))
```

```
[0.33333333 0.33333333 0.33333333]
[0.33333333 0.33333333 0.33333333]
[0.33333333 0.33333333 0.33333333]
```

[15]:
```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(90, 4) (60, 4) (90,) (60,)
```

normalization

[16]:
```
from sklearn.preprocessing import StandardScaler
```

[17]:
```
norm = StandardScaler()
norm.fit(X_train)

X_train_norm = norm.transform(X_train)
X_test_norm = norm.transform(X_test)
```

0.2 Classifiers

Classifiers usually have a: - `fit` method to train them on training data - `predict` method to validate/test them on validation/test data

0.3 KNN

[18]:
```
from sklearn.neighbors import KNeighborsClassifier
```

[19]:
```
clf = KNeighborsClassifier(n_neighbors=5, metric="euclidean", weights="uniform")
clf.fit(X_train_norm, y_train)
```

[19]:
```
KNeighborsClassifier(metric='euclidean')
```

[20]:
```
# predict: Predict the class labels for the provided data.
y_test_pred = clf.predict(X_test_norm)
y_test_pred
```

[20]:
```
array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2, 1, 0, 2, 2, 1, 0,
       2, 0, 1, 1, 2, 0, 1, 2, 0, 2, 1, 1, 2, 0, 2, 1, 1, 2, 1, 0, 1, 2,
       2, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 2, 0, 1, 1])
```

[21]:
```
# y_test contains the target labels
y_test
```

[21]:
```
array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2, 1, 0, 2, 2, 1, 0,
       2, 0, 1, 1, 2, 0, 1, 2, 0, 2, 1, 1, 2, 0, 2, 1, 1, 2, 1, 0, 1, 2,
       2, 1, 1, 0, 0, 0, 0, 2, 1, 0, 0, 1, 2, 0, 1, 1])
```

[22]:
```
print("Accuracy:", accuracy_score(y_test, y_test_pred))
```

```
Accuracy: 0.9833333333333333
```

[23]:
```
# score: Return the mean accuracy on the given test data and labels.
clf.score(X_test_norm, y_test)
```

[23]: 0.9833333333333333

[24]:
```
# KNeighborsClassifier.score is doing this
(y_test_pred == y_test).sum() / len(y_test)
```

[24]: 0.9833333333333333

Performance evaluation

[25]:
```
# print("F1:", f1_score(y_test, y_test_pred))
```

[26]:
```
print("F1:", f1_score(y_test, y_test_pred, average="macro"))
```

```
F1: 0.9833229101521784
```

[27]:
```
print("F1:", f1_score(y_test, y_test_pred, average="micro"))
```

```
F1: 0.9833333333333333
```

[28]:
```
print("F1:", f1_score(y_test, y_test_pred, labels=[1], average="micro"))
```

```
F1: 0.975609756097561
```

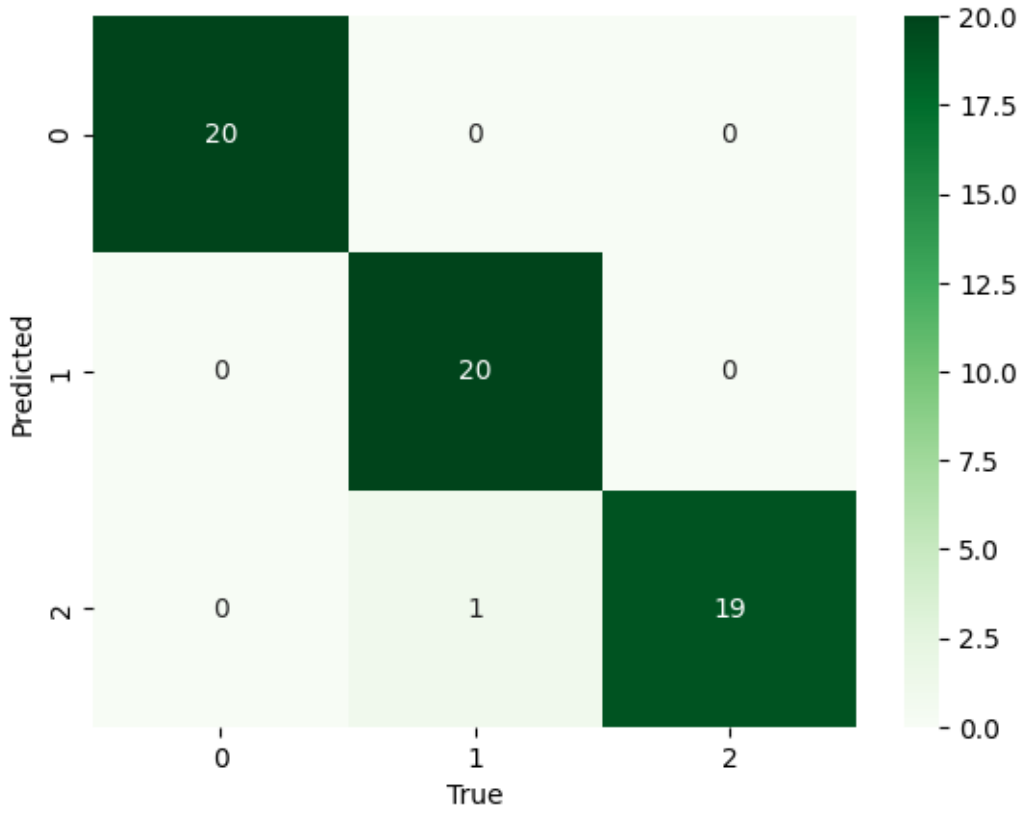
[29]:
```
print("F1:", f1_score(y_test, y_test_pred, labels=[1], average="macro"))
```

```
F1: 0.975609756097561
```

[30]:
```
print(classification_report(y_test, y_test_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        20
           1       0.95      1.00      0.98        20
           2       1.00      0.95      0.97        20

    accuracy                           0.98        60
   macro avg       0.98      0.98      0.98        60
weighted avg       0.98      0.98      0.98        60
```

[31]:
```
cf = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cf, annot=True, cmap="Greens")
plt.xlabel("True")
plt.ylabel("Predicted")
plt.show()
```



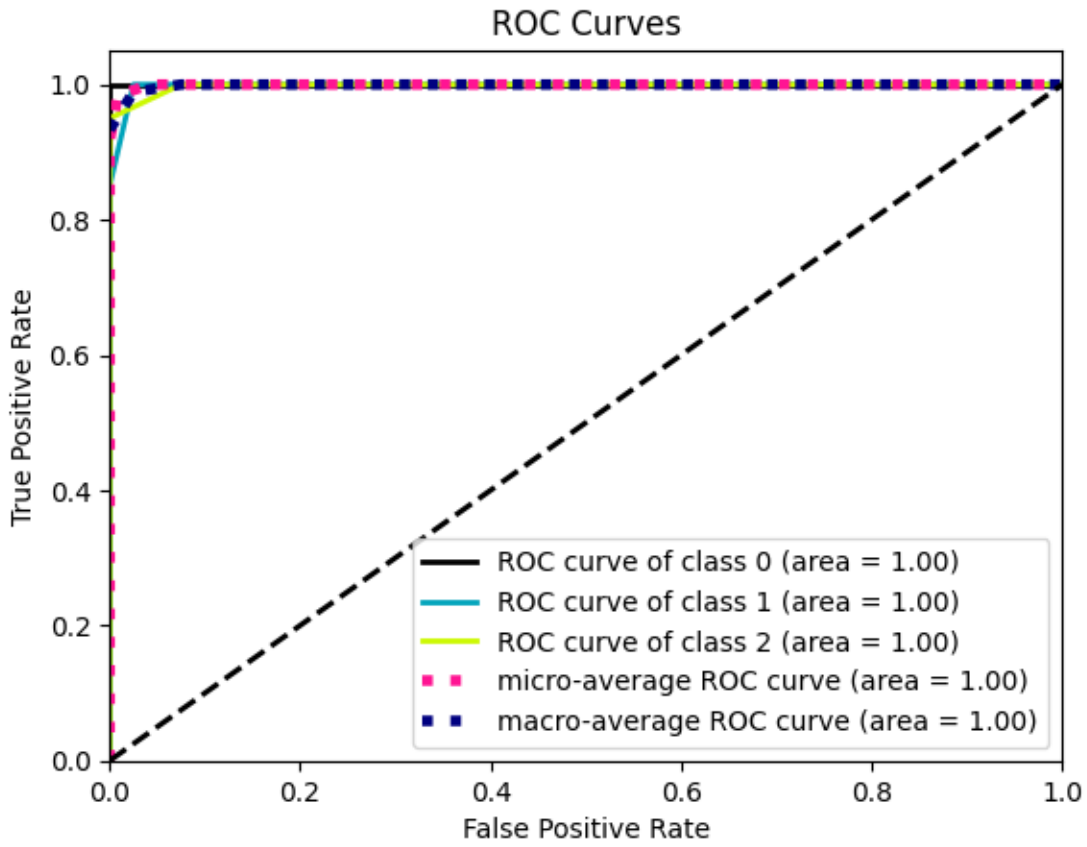
[32]:
```
# Return probability estimates for the test data X.
y_test_pred_proba = clf.predict_proba(X_test_norm)
y_test_pred_proba[0:10]
```

[32]:
```
array([[0. , 0.2, 0.8],
       [0. , 0. , 1. ],
       [1. , 0. , 0. ],
       [0. , 0. , 1. ],
       [1. , 0. , 0. ],
       [1. , 0. , 0. ],
       [1. , 0. , 0. ],
       [0. , 0. , 1. ],
       [0. , 1. , 0. ],
       [0. , 1. , 0. ]])
```

[33]:
```
y_test_pred[0:10]
```

[33]:
```
array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1])
```

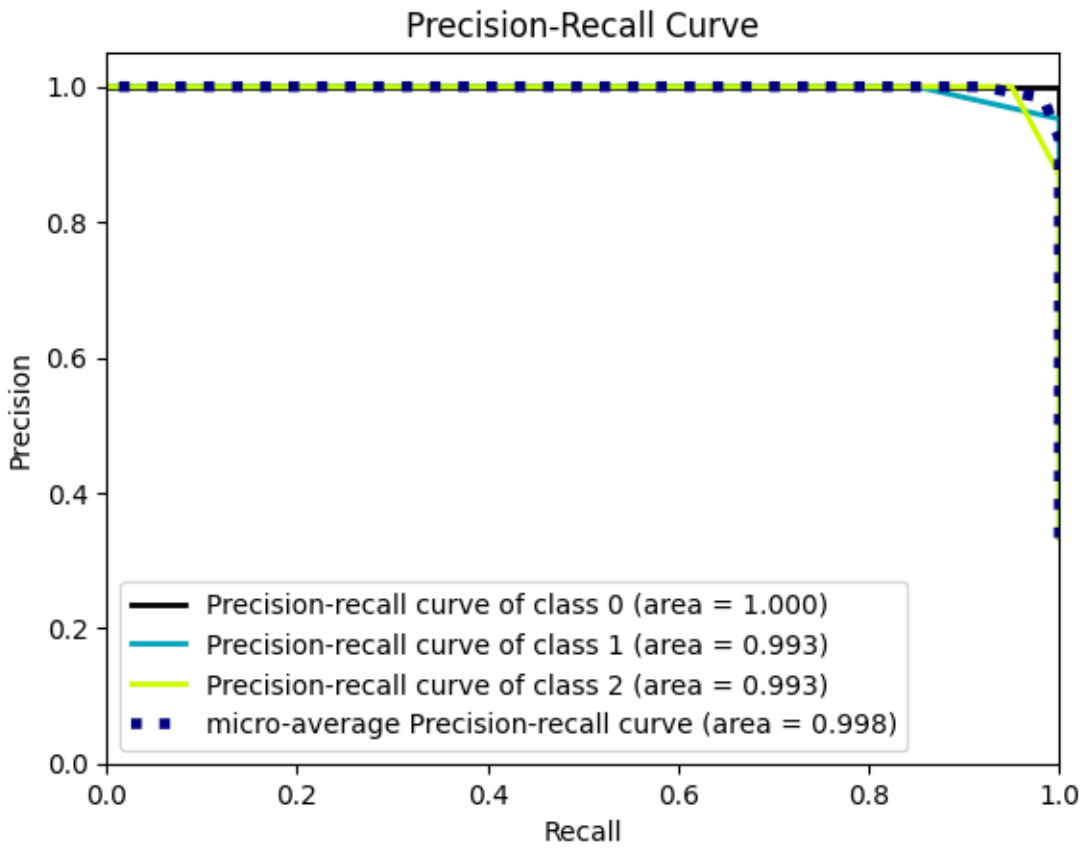
[34]:
```
# https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html
plot_roc(y_test, y_test_pred_proba)
plt.show()
```



[35]:
```
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.
↪roc_auc_score.html
roc_auc_score(y_test, y_test_pred_proba, multi_class="ovr", average="macro")
```

[35]:
```
0.9987499999999999
```

[36]:
```
plot_precision_recall(y_test, y_test_pred_proba)
plt.show()
```



Repeated Holdout

[37]:
```
N = 50
err = 0

for i in range(N):
    # stratified holdout
    X_rh_train, X_rh_test, y_rh_train, y_rh_test = train_test_split(X, y,␣
 ↪test_size=0.4, stratify=y)

    # normalize train set
    norm.fit(X_rh_train)
    X_rh_train_norm = norm.transform(X_rh_train)
    X_rh_test_norm = norm.transform(X_rh_test)

    # initialize and fit classifier
    clf = KNeighborsClassifier(n_neighbors=5, metric="euclidean",␣
 ↪weights="uniform")
    clf.fit(X_rh_train_norm, y_rh_train)
```

```
    # computing error
    acc = clf.score(X_rh_test_norm, y_rh_test)
    err += 1 - acc

print("Overall error estimate:", err/N)
```

```
Overall error estimate: 0.04866666666666667
```

[37]:

Cross-validation https://scikit-learn.org/stable/modules/cross_validation.html

[38]:
```
from sklearn.model_selection import cross_val_score
k = 10
```

[39]:
```
# initialize classifier
clf = KNeighborsClassifier(n_neighbors=5, metric="euclidean", weights="uniform")

scores = cross_val_score(clf, X_train_norm, y_train, cv=k)
scores
```

[39]:
```
array([0.88888889, 1.        , 1.        , 1.        , 0.77777778,
       1.        , 0.55555556, 0.88888889, 1.        , 1.        ])
```

[40]:
```
print("Overall error estimate:", 1 - scores.mean())
```

```
Overall error estimate: 0.0888888888888889
```

[41]:
```
print('Accuracy: %0.4f (+/- %0.2f)' % (scores.mean(), scores.std()))
```

```
Accuracy: 0.9111 (+/- 0.14)
```

[42]:
```
# scoring default is accuracy
cross_val_score(clf, X_train_norm, y_train, cv=k, scoring='f1_macro')
```

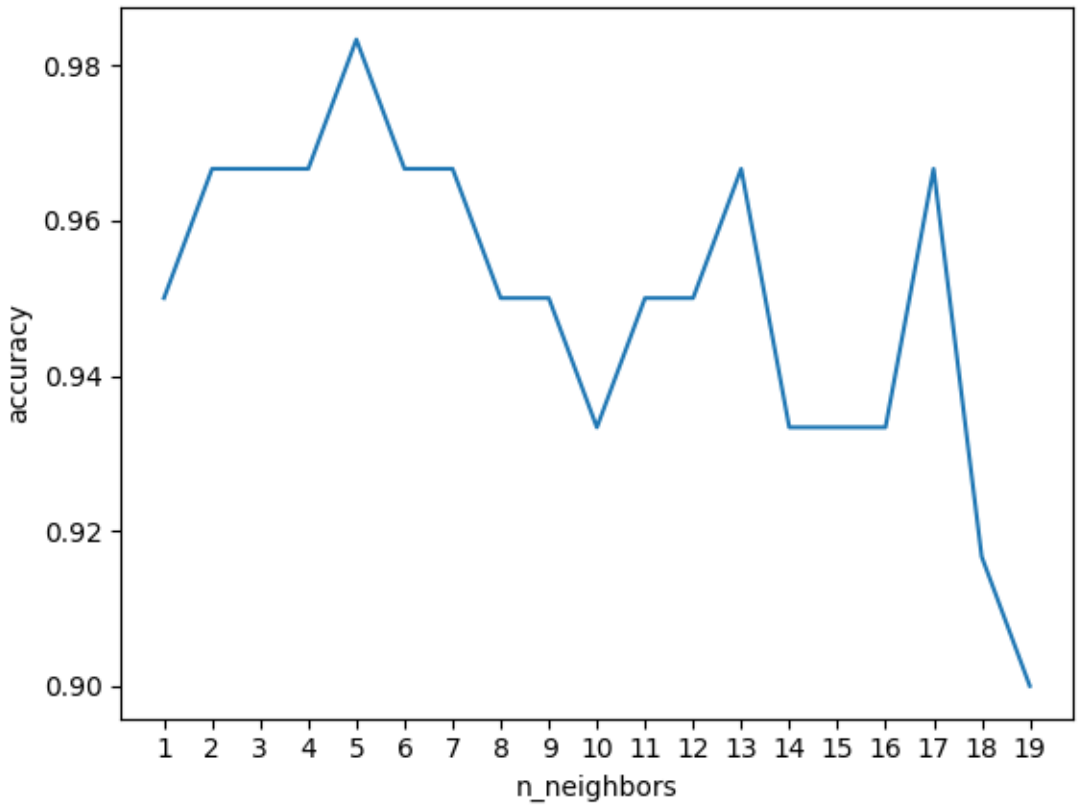
[42]:
```
array([0.88571429, 1.        , 1.        , 1.        , 0.77777778,
       1.        , 0.55555556, 0.88571429, 1.        , 1.        ])
```

[42]:

0.3.1 Hyperparameters Tuning

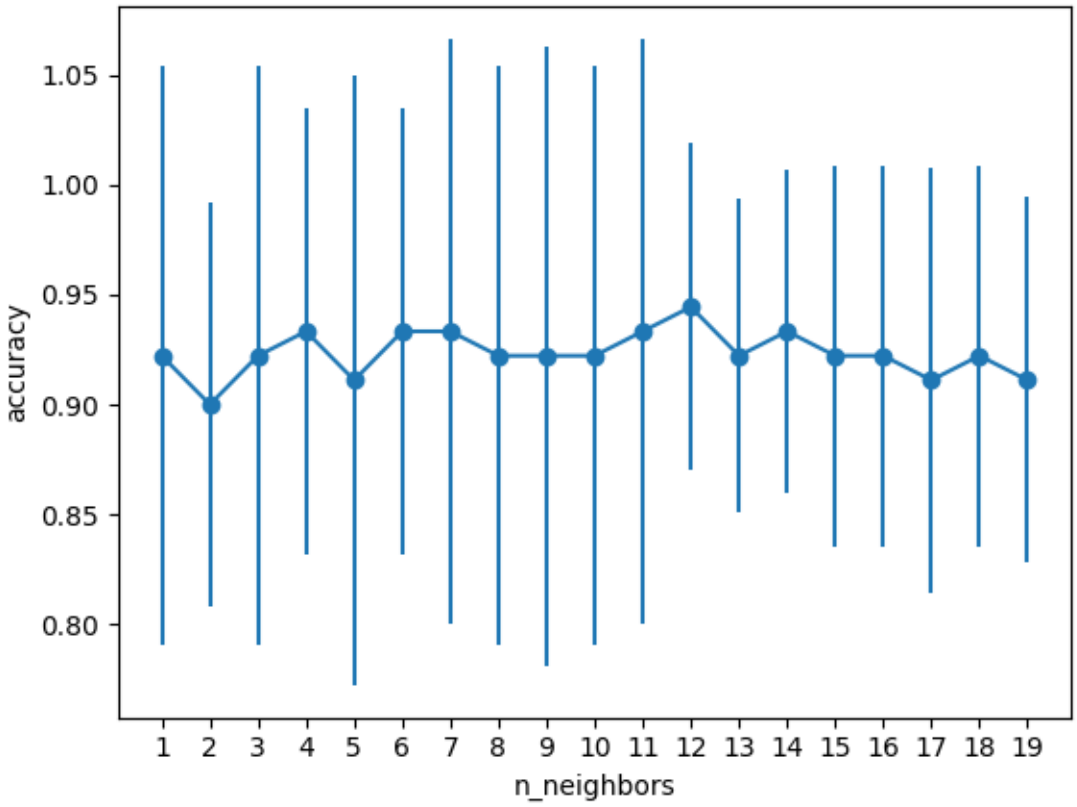
[43]:
```
n_neighbors = range(1,20)
scores = list()

for n in n_neighbors:
```

```
    clf = KNeighborsClassifier(n_neighbors=n, metric="euclidean",␣
 ↪weights="uniform")
    clf.fit(X_train_norm, y_train)
    scores.append(clf.score(X_test_norm, y_test))

plt.plot(scores)
plt.xticks(range(len(n_neighbors)), n_neighbors)
plt.xlabel("n_neighbors")
plt.ylabel("accuracy")
plt.show()
```



[44]:
```
n_neighbors = range(1,20)
avg_scores = list()
std_scores = list()

for n in n_neighbors:
    clf = KNeighborsClassifier(n_neighbors=n, metric="euclidean",␣
 ↪weights="uniform")
    scores = cross_val_score(clf, X_train_norm, y_train, cv=k)
    avg_scores.append(np.mean(scores))
    std_scores.append(np.std(scores))
```

```
#plt.plot(avg_scores)
plt.errorbar(range(len(n_neighbors)), y=avg_scores, yerr=std_scores, marker='o')
plt.xticks(range(len(n_neighbors)), n_neighbors)
plt.xlabel("n_neighbors")
plt.ylabel("accuracy")
plt.show()
```



[45]:
```
clf = KNeighborsClassifier(n_neighbors=12, metric="euclidean",␣
 ↪weights="uniform")
clf.fit(X_train_norm, y_train)
y_test_pred = clf.predict(X_test_norm)
print("Accuracy:", accuracy_score(y_test, y_test_pred))
```

```
Accuracy: 0.95
```

Grid Search

[46]:
```
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
```

[47]:
```
%%time
param_grid = {
    "n_neighbors": (np.arange(1, X_train.shape[0]//2)),
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "cityblock"],
}

grid = GridSearchCV(
    KNeighborsClassifier(),
    param_grid=param_grid,
    cv=RepeatedStratifiedKFold(random_state=0),
    n_jobs=-1,
    refit=True,
    # verbose=2
)

#param_distributions=param_grid,

grid.fit(X_train_norm, y_train)
clf = grid.best_estimator_
```

```
CPU times: user 2.72 s, sys: 73.8 ms, total: 2.79 s
Wall time: 41.9 s
```

[48]:
```
print(grid.best_params_, grid.best_score_)
```

```
{'metric': 'euclidean', 'n_neighbors': 14, 'weights': 'distance'}
0.9466666666666665
```

[49]:
```
y_test_pred = clf.predict(X_test_norm)
print("Accuracy:", accuracy_score(y_test, y_test_pred))
```

```
Accuracy: 0.9666666666666667
```

[50]:
```
clf.score(X_test_norm, y_test)
```

[50]:
```
0.9666666666666667
```

[51]:
```
# grid.cv_results_
```

[52]:
```
results = pd.DataFrame(grid.cv_results_)
results
```

[52]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	0.004583	0.003372	0.013991	0.009098	
1	0.003988	0.002896	0.005595	0.002973	
2	0.002931	0.002279	0.009794	0.005777	
3	0.002762	0.001809	0.003857	0.001688	

```
4         0.002036      0.001417         0.008432        0.004344
..             …             …                …             …
171       0.001093      0.000549         0.001989        0.001017
172       0.001003      0.000344         0.001336        0.000160
173       0.001200      0.000989         0.001999        0.000790
174       0.001051      0.000410         0.001465        0.000550
175       0.001002      0.000256         0.001812        0.000439

    param_metric  param_n_neighbors param_weights  \
0      euclidean                  1       uniform
1      euclidean                  1      distance
2      euclidean                  2       uniform
3      euclidean                  2      distance
4      euclidean                  3       uniform
..           …                  …             …
171    cityblock                 42      distance
172    cityblock                 43       uniform
173    cityblock                 43      distance
174    cityblock                 44       uniform
175    cityblock                 44      distance

                                                 params  split0_test_score  \
0    {'metric': 'euclidean', 'n_neighbors': 1, 'wei…           0.888889
1    {'metric': 'euclidean', 'n_neighbors': 1, 'wei…           0.888889
2    {'metric': 'euclidean', 'n_neighbors': 2, 'wei…           0.888889
3    {'metric': 'euclidean', 'n_neighbors': 2, 'wei…           0.888889
4    {'metric': 'euclidean', 'n_neighbors': 3, 'wei…           0.888889
..                                                 …                 …
171  {'metric': 'cityblock', 'n_neighbors': 42, 'we…           0.833333
172  {'metric': 'cityblock', 'n_neighbors': 43, 'we…           0.888889
173  {'metric': 'cityblock', 'n_neighbors': 43, 'we…           0.833333
174  {'metric': 'cityblock', 'n_neighbors': 44, 'we…           0.833333
175  {'metric': 'cityblock', 'n_neighbors': 44, 'we…           0.833333

     split1_test_score  …  split43_test_score  split44_test_score  \
0             0.944444  …            0.944444            0.833333
1             0.944444  …            0.944444            0.833333
2             0.888889  …            0.944444            0.888889
3             0.944444  …            0.944444            0.833333
4             0.888889  …            0.944444            0.888889
..                 …  …                …                 …
171           0.944444  …            0.944444            0.833333
172           0.888889  …            0.888889            0.777778
173           0.944444  …            0.944444            0.833333
174           0.833333  …            0.888889            0.666667
175           0.944444  …            0.944444            0.833333
```

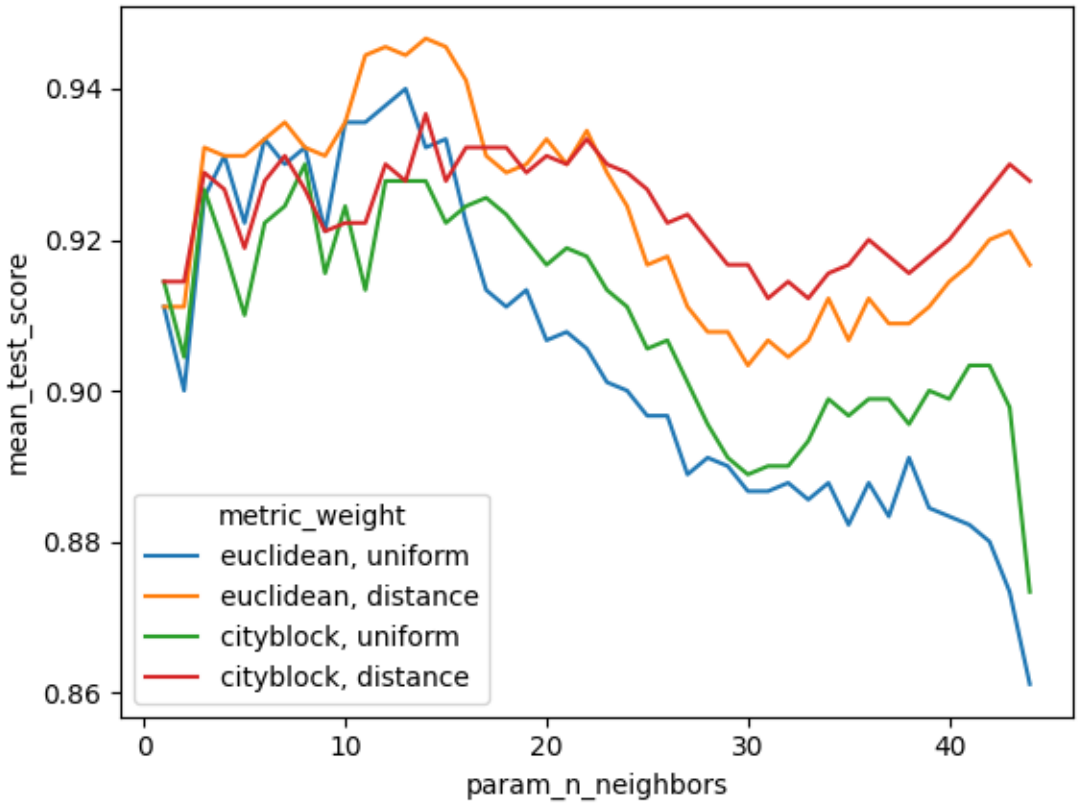
```
     split45_test_score  split46_test_score  split47_test_score  \
0              1.000000            0.944444            0.944444
1              1.000000            0.944444            0.944444
2              0.888889            0.944444            0.944444
3              1.000000            0.944444            0.944444
4              0.944444            0.944444            0.944444
..                  …                   …                   …
171            0.944444            0.833333            0.944444
172            1.000000            0.833333            0.777778
173            1.000000            0.833333            0.944444
174            0.888889            0.833333            0.944444
175            0.944444            0.833333            0.944444

     split48_test_score  split49_test_score  mean_test_score  std_test_score  \
0              0.833333            0.944444         0.911111        0.060858
1              0.833333            0.944444         0.911111        0.060858
2              0.833333            0.944444         0.900000        0.056656
3              0.833333            0.944444         0.911111        0.060858
4              0.888889            1.000000         0.925556        0.056229
..                  …                   …                …               …
171            0.944444            0.944444         0.926667        0.063285
172            0.888889            0.888889         0.897778        0.058119
173            0.944444            0.944444         0.930000        0.062667
174            0.888889            0.833333         0.873333        0.069424
175            0.944444            0.944444         0.927778        0.061111

     rank_test_score
0                113
1                113
2                140
3                113
4                 59
..                …
171               57
172              147
173               34
174              174
175               48

[176 rows x 61 columns]
```

[53]:
```
results["metric_weight"] = results["param_metric"] + ", " +␣
↪results["param_weights"]
```

[54]:
```
sns.lineplot(
    data=results, x="param_n_neighbors", y="mean_test_score",␣
↪hue="metric_weight"
```

```
)
```

[54]: <Axes: xlabel='param_n_neighbors', ylabel='mean_test_score'>



[54]:

0.3.2 Naive Bayes

[55]:
```
from sklearn.naive_bayes import GaussianNB, CategoricalNB
```

Gaussian

[56]:
```
clf = GaussianNB()
```

[57]:
```
%%time
clf.fit(X_train, y_train)
```

```
CPU times: user 2.6 ms, sys: 0 ns, total: 2.6 ms
Wall time: 2.5 ms
```

[57]: GaussianNB()

[58]:
```
y_pred = clf.predict(X_test)
y_pred
```

[58]:
```
array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2, 1, 0, 2, 2, 1, 0,
       2, 0, 1, 1, 2, 0, 1, 2, 0, 2, 1, 1, 2, 0, 2, 1, 1, 2, 1, 0, 1, 2,
       2, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 2, 0, 1, 2])
```

[59]:
```
print(classification_report(y_test, y_pred))
```

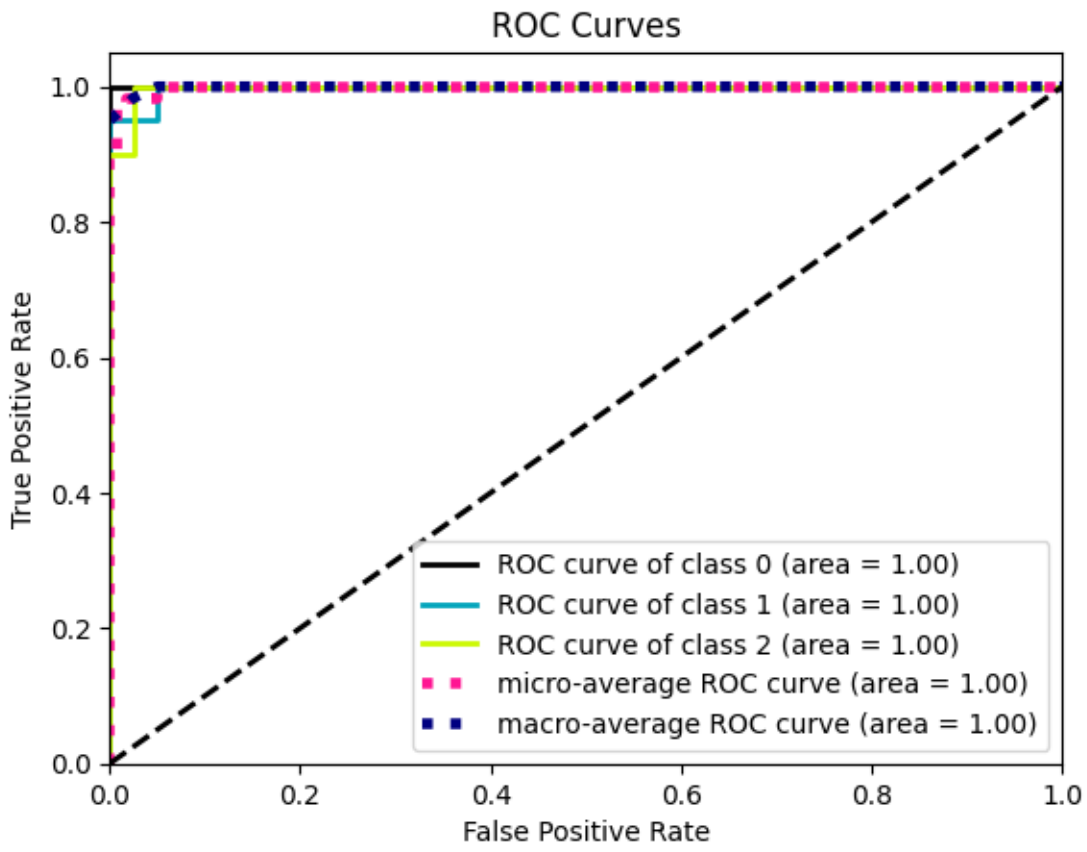
```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        20
           1       0.95      0.95      0.95        20
           2       0.95      0.95      0.95        20

    accuracy                           0.97        60
   macro avg       0.97      0.97      0.97        60
weighted avg       0.97      0.97      0.97        60
```

[60]:
```
clf.predict_proba(X_test)
```

[60]:
```
array([[3.05055218e-156, 1.98469602e-002, 9.80153040e-001],
       [2.50142195e-277, 3.75932821e-012, 1.00000000e+000],
       [1.00000000e+000, 2.74048199e-017, 2.82825801e-026],
       [3.77047257e-248, 6.87918788e-010, 9.99999999e-001],
       [1.00000000e+000, 1.81924966e-016, 2.52343415e-026],
       [1.00000000e+000, 5.84024273e-017, 8.14494337e-027],
       [1.00000000e+000, 1.12344252e-016, 5.92512274e-026],
       [6.19173845e-176, 4.67941390e-004, 9.99532059e-001],
       [1.10238158e-032, 9.99999857e-001, 1.43256972e-007],
       [7.56032487e-075, 9.99968118e-001, 3.18821921e-005],
       [1.00000000e+000, 1.66546269e-010, 2.54675324e-019],
       [3.73265040e-154, 6.58305961e-002, 9.34169404e-001],
       [0.00000000e+000, 1.70481203e-012, 1.00000000e+000],
       [1.00000000e+000, 6.04531187e-012, 4.37830158e-020],
       [1.35690609e-092, 9.96128400e-001, 3.87159969e-003],
       [7.97250163e-267, 3.29344440e-010, 1.00000000e+000],
       [1.86513942e-098, 9.78842746e-001, 2.11572540e-002],
       [1.00000000e+000, 4.88720961e-019, 3.68247362e-028],
       [3.59336710e-162, 1.86623554e-002, 9.81337645e-001],
       [7.11242895e-161, 3.42090433e-002, 9.65790957e-001],
       [1.60527365e-059, 9.99978974e-001, 2.10262412e-005],
       [1.00000000e+000, 1.64906577e-013, 9.88604933e-023],
       [3.34150260e-200, 1.31058957e-006, 9.99998689e-001],
       [1.00000000e+000, 1.98789412e-017, 5.32896313e-026],
       [6.47625569e-079, 9.99895656e-001, 1.04343789e-004],
```

```
       [5.29919058e-075, 9.99975461e-001, 2.45393824e-005],
       [1.83093152e-230, 2.54506289e-007, 9.99999745e-001],
       [1.00000000e+000, 5.44711014e-017, 2.03232679e-025],
       [1.01049595e-112, 9.55075200e-001, 4.49247999e-002],
       [5.37442739e-197, 6.84956056e-006, 9.99993150e-001],
       [1.00000000e+000, 5.19559717e-017, 5.25698390e-026],
       [1.49048596e-231, 1.04650690e-006, 9.99998953e-001],
       [9.09721319e-076, 9.99853438e-001, 1.46562275e-004],
       [1.18521797e-117, 7.83718534e-001, 2.16281466e-001],
       [6.39795698e-170, 4.39554222e-004, 9.99560446e-001],
       [1.00000000e+000, 2.48282507e-017, 6.08647478e-027],
       [4.53144050e-231, 1.86017095e-008, 9.99999981e-001],
       [7.95618521e-107, 9.36508854e-001, 6.34911462e-002],
       [9.78193383e-052, 9.99999547e-001, 4.52961673e-007],
       [1.14590516e-144, 1.41113424e-001, 8.58886576e-001],
       [1.74966107e-113, 7.82539255e-001, 2.17460745e-001],
       [1.00000000e+000, 4.34259424e-017, 1.51767006e-026],
       [9.46439464e-108, 9.94182260e-001, 5.81774047e-003],
       [7.27856849e-186, 2.70725704e-003, 9.97292743e-001],
       [2.69165369e-190, 7.44082653e-004, 9.99255917e-001],
       [1.93165252e-110, 8.30626172e-001, 1.69373828e-001],
       [2.57768066e-078, 9.99852564e-001, 1.47435570e-004],
       [1.00000000e+000, 1.44977761e-017, 1.07574272e-026],
       [1.00000000e+000, 1.97922847e-016, 2.11881014e-025],
       [1.00000000e+000, 8.87084357e-017, 6.08037091e-026],
       [1.00000000e+000, 1.63263330e-016, 3.04083318e-026],
       [4.03680708e-162, 6.49790168e-001, 3.50209832e-001],
       [1.39614388e-087, 9.99941983e-001, 5.80174634e-005],
       [1.00000000e+000, 3.80451594e-015, 1.43324210e-023],
       [1.00000000e+000, 6.37708578e-014, 7.98165218e-022],
       [1.51696008e-096, 9.99487070e-001, 5.12930491e-004],
       [7.60262226e-200, 4.10479572e-006, 9.99995895e-001],
       [1.00000000e+000, 1.99771588e-017, 2.03445104e-027],
       [1.71249185e-067, 9.99995530e-001, 4.46979378e-006],
       [4.82476701e-145, 7.05726601e-002, 9.29427340e-001]])
```

[61]:
```
# https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html
plot_roc(y_test, clf.predict_proba(X_test))
plt.show()
print(roc_auc_score(y_test, clf.predict_proba(X_test), multi_class="ovr",␣
 ↪average="macro"))
```




```
0.9983333333333334
```

0.3.3 Wine dataset

These data are the results of a chemical analysis of wines grown in the same region in Italy but derived from 3 different cultivars. The analysis determined the quantities of 13 constituents found in each of the three types of wines.

[62]:
```
from sklearn.datasets import load_wine

d = load_wine(as_frame=True)
df_orig = d["data"]
y = np.array(d["target"])

df_orig.head()
```

[62]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	

```
3    14.37        1.95  2.50               16.8      113.0           3.85
4    13.24        2.59  2.87               21.0      118.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
0        3.06                  0.28             2.29             5.64  1.04
1        2.76                  0.26             1.28             4.38  1.05
2        3.24                  0.30             2.81             5.68  1.03
3        3.49                  0.24             2.18             7.80  0.86
4        2.69                  0.39             1.82             4.32  1.04

   od280/od315_of_diluted_wines  proline
0                          3.92   1065.0
1                          3.40   1050.0
2                          3.17   1185.0
3                          3.45   1480.0
4                          2.93    735.0
```

[63]:
```
# adding a fake categorical variable to see how to deal with it
cat_variable_1_values = ["red", "blue", "green"]
df_orig["color"] = [
    cat_variable_1_values[np.random.randint(0, len(cat_variable_1_values))]
    for _ in range(len(df_orig))
]

# creating an ordinal variable
df_orig["alcalinity_of_ash_binned"] = pd.qcut(
    df_orig["alcalinity_of_ash"], q=4, labels=False
)

X_orig = df_orig.values
df_orig.head()

# one-hot encoding of categorical data
categorical_cols = ["color"]

df = pd.get_dummies(df_orig, columns=categorical_cols)
X = df.values
df.head()
```

[63]:
```
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80
3    14.37        1.95  2.50               16.8      113.0           3.85
4    13.24        2.59  2.87               21.0      118.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
```

```
0        3.06                  0.28             2.29              5.64  1.04
1        2.76                  0.26             1.28              4.38  1.05
2        3.24                  0.30             2.81              5.68  1.03
3        3.49                  0.24             2.18              7.80  0.86
4        2.69                  0.39             1.82              4.32  1.04

   od280/od315_of_diluted_wines  proline  alcalinity_of_ash_binned  \
0                          3.92   1065.0                         0
1                          3.40   1050.0                         0
2                          3.17   1185.0                         1
3                          3.45   1480.0                         0
4                          2.93    735.0                         2

   color_blue  color_green  color_red
0       False         True      False
1       False         True      False
2       False        False       True
3       False         True      False
4       False         True      False
```

[64]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=random_state
)
```

[65]:
```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(124, 17) (54, 17) (124,) (54,)
```

Gaussian NB

[66]:
```
clf = GaussianNB()
```

[67]:
```
%%time
clf.fit(X_train, y_train)
```

```
CPU times: user 2.28 ms, sys: 758 µs, total: 3.04 ms
Wall time: 3.03 ms
```

[67]:
```
GaussianNB()
```

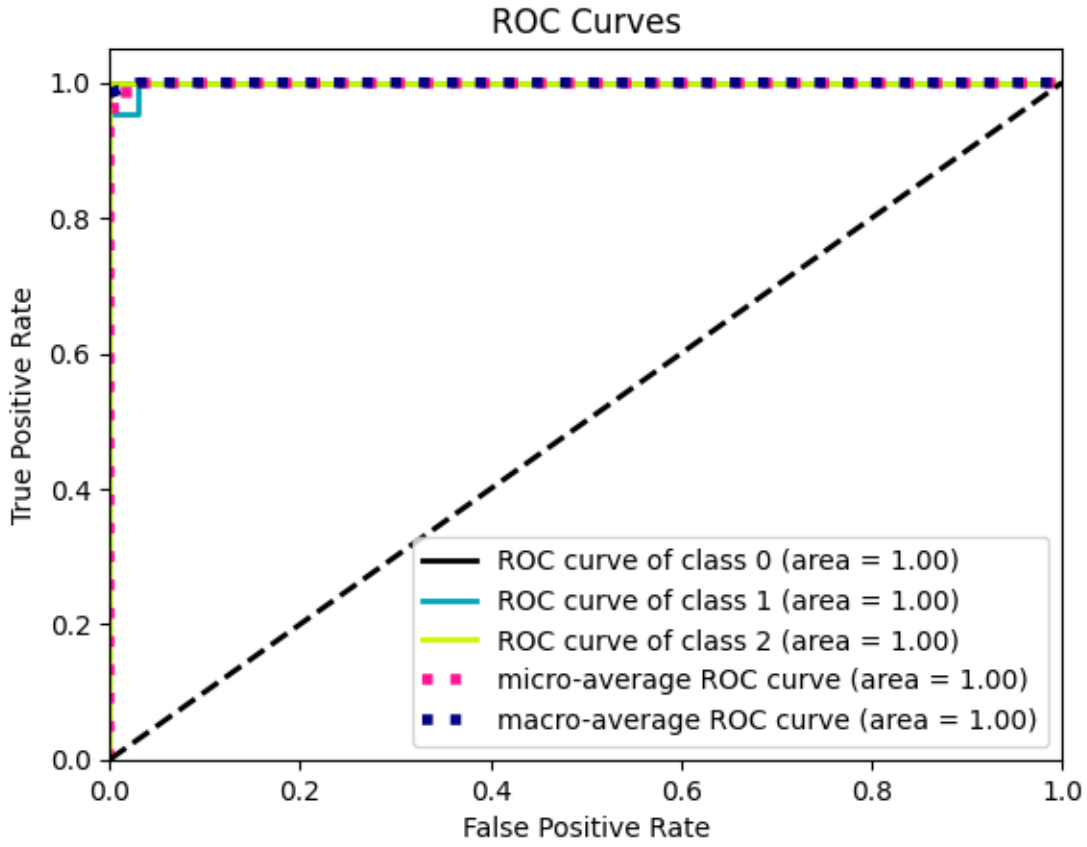
[68]:
```
y_pred = clf.predict(X_test)
```

[69]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.94      0.97        18
           1       0.95      0.95      0.95        21
```

```
           2       0.94      1.00      0.97        15

    accuracy                           0.96        54
   macro avg       0.96      0.97      0.96        54
weighted avg       0.96      0.96      0.96        54
```

[70]:
```
# https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html
plot_roc(y_test, clf.predict_proba(X_test))
plt.show()
print(roc_auc_score(y_test, clf.predict_proba(X_test), multi_class="ovr",␣
 ↪average="macro"))
```




```
0.9995189995189996
```

[70]:

Categorical

[71]:
```
non_cat_columns = [
    "alcohol",
    "malic_acid",
    "ash",
    "alcalinity_of_ash",
    "magnesium",
    "total_phenols",
    "flavanoids",
    "nonflavanoid_phenols",
    "proanthocyanins",
    "color_intensity",
    "hue",
    "od280/od315_of_diluted_wines",
    "proline",
]

X_noncat = df[non_cat_columns].values

X_train_noncat, X_test_noncat, y_train_noncat, y_test_noncat = train_test_split(
    X_noncat, y, test_size=0.3, stratify=y, random_state=random_state
)

# train and test set should be binned separately
X_train_cat = list()
for column_idx in range(X_train_noncat.shape[1]):
    X_train_cat.append(pd.qcut(X_train_noncat[:, column_idx], q=4,␣
 ↪labels=False))
X_train_cat = np.array(X_train_cat).T

X_test_cat = list()
for column_idx in range(X_test_noncat.shape[1]):
    X_test_cat.append(pd.qcut(X_test_noncat[:, column_idx], q=4, labels=False))
X_test_cat = np.array(X_test_cat).T

print(X_train_cat.shape, X_test_cat.shape)
```

```
(124, 13) (54, 13)
```

[72]:
```
X_train_cat
```

[72]:
```
array([[2, 3, 1, …, 1, 1, 1],
       [3, 0, 3, …, 3, 2, 3],
       [3, 0, 3, …, 3, 1, 3],
       …,
       [3, 3, 0, …, 0, 0, 0],
       [0, 0, 1, …, 2, 2, 2],
       [1, 3, 2, …, 3, 2, 0]])
```

[73]:
```
clf = CategoricalNB()
clf.fit(X_train_cat, y_train_noncat)
```

[73]:
```
CategoricalNB()
```

[74]:
```
X_train_cat
```

[74]:
```
array([[2, 3, 1, …, 1, 1, 1],
       [3, 0, 3, …, 3, 2, 3],
       [3, 0, 3, …, 3, 1, 3],
       …,
       [3, 3, 0, …, 0, 0, 0],
       [0, 0, 1, …, 2, 2, 2],
       [1, 3, 2, …, 3, 2, 0]])
```

[75]:
```
y_pred = clf.predict(X_test_cat)
```

[76]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        18
           1       1.00      0.90      0.95        21
           2       0.88      1.00      0.94        15

    accuracy                           0.96        54
   macro avg       0.96      0.97      0.96        54
weighted avg       0.97      0.96      0.96        54
```

1 Binary Classification Example

[77]:
```
from scikitplot.metrics import plot_cumulative_gain, plot_lift_curve
# plot_cumulative_gain and plot_lift_curve only work in a binary classification␣
 ↪case
```

[78]:
```
from sklearn.datasets import load_breast_cancer
```

[79]:
```
X, y = load_breast_cancer(return_X_y=True)
```

[79]:
```

```

[80]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=random_state
)
```

[81]:
```
clf = GaussianNB()
```

[82]:
```
clf.fit(X_train, y_train)
```

[82]:
```
GaussianNB()
```

[83]:
```
y_test_pred_proba = clf.predict_proba(X_test)
y_test_pred_proba
```

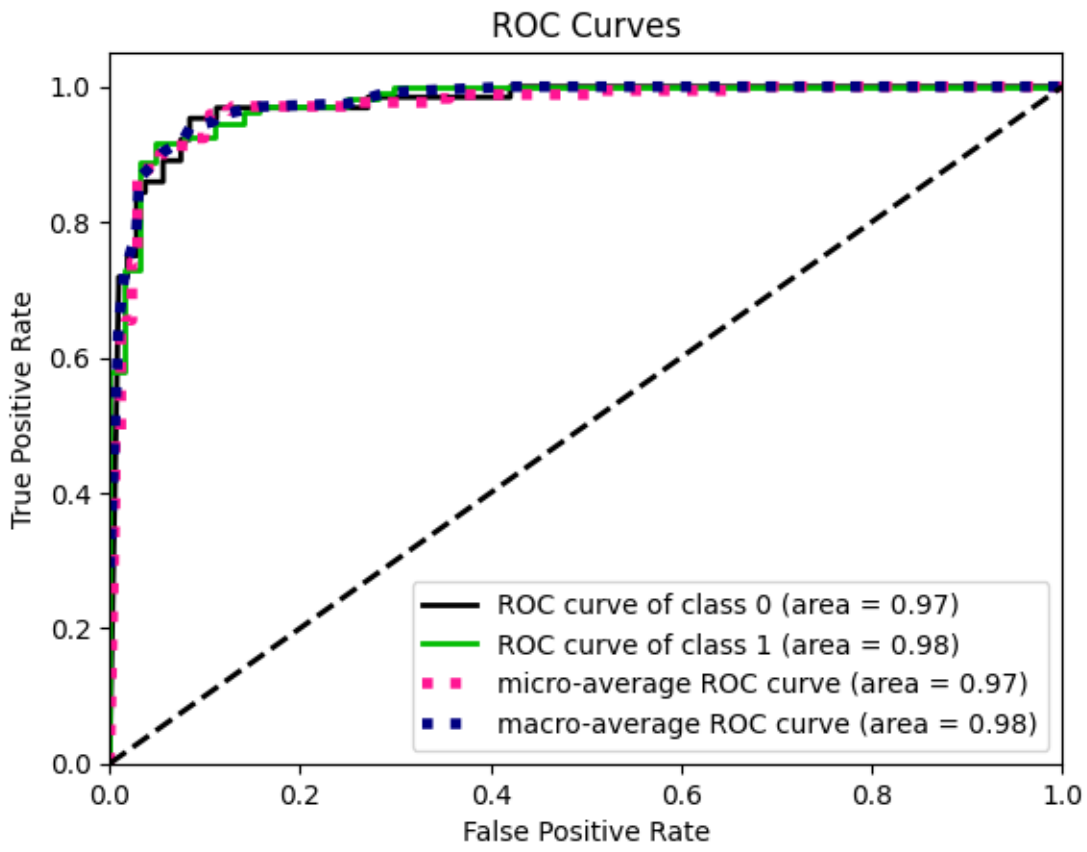
[83]:
```
array([[1.00000000e+000, 9.60798304e-067],
       [3.28452382e-012, 1.00000000e+000],
       [1.00000000e+000, 1.37358518e-029],
       [9.07175634e-001, 9.28243663e-002],
       [1.00000000e+000, 1.13066573e-026],
       [4.86470493e-011, 1.00000000e+000],
       [1.00000000e+000, 1.85354720e-229],
       [1.00000000e+000, 2.24061507e-034],
       [1.00000000e+000, 7.64991022e-020],
       [1.00000000e+000, 5.98723466e-025],
       [1.55676568e-010, 1.00000000e+000],
       [4.87665583e-012, 1.00000000e+000],
       [2.51652415e-014, 1.00000000e+000],
       [3.35003877e-015, 1.00000000e+000],
       [1.00000000e+000, 7.27215795e-019],
       [1.00000000e+000, 5.01138848e-047],
       [1.00000000e+000, 3.84869046e-038],
       [5.11581566e-005, 9.99948842e-001],
       [1.00000000e+000, 5.99928618e-030],
       [1.00000000e+000, 8.93234543e-263],
       [9.23899517e-016, 1.00000000e+000],
       [1.22947094e-013, 1.00000000e+000],
       [4.60035794e-013, 1.00000000e+000],
       [1.00000000e+000, 2.83284900e-302],
       [1.00000000e+000, 1.92821921e-054],
       [4.94411999e-005, 9.99950559e-001],
       [6.63793002e-012, 1.00000000e+000],
       [1.00000000e+000, 4.93986527e-097],
       [5.84717888e-014, 1.00000000e+000],
       [1.00000000e+000, 1.59716799e-098],
       [8.87809741e-001, 1.12190259e-001],
       [4.04806236e-013, 1.00000000e+000],
       [9.99999683e-001, 3.16530138e-007],
       [3.93236381e-014, 1.00000000e+000],
       [1.25847936e-014, 1.00000000e+000],
       [1.00000000e+000, 1.61535881e-014],
       [6.30346164e-008, 9.99999937e-001],
       [1.00000000e+000, 9.79137325e-237],
```

```
[3.98832803e-019, 1.00000000e+000],
[9.96202524e-001, 3.79747602e-003],
[6.37533647e-001, 3.62466353e-001],
[1.00000000e+000, 3.47386136e-073],
[8.65947829e-011, 1.00000000e+000],
[1.00000000e+000, 2.16768230e-012],
[4.79603200e-010, 1.00000000e+000],
[1.00000000e+000, 8.70229084e-065],
[1.00000000e+000, 1.49902651e-028],
[2.13772341e-016, 1.00000000e+000],
[2.42737760e-019, 1.00000000e+000],
[1.27400030e-016, 1.00000000e+000],
[4.08428604e-010, 1.00000000e+000],
[1.00000000e+000, 3.06395512e-201],
[6.37932236e-018, 1.00000000e+000],
[1.00000000e+000, 8.43935225e-045],
[2.87486870e-005, 9.99971251e-001],
[1.68740960e-015, 1.00000000e+000],
[9.79395040e-001, 2.06049605e-002],
[6.30316863e-008, 9.99999937e-001],
[1.95612618e-014, 1.00000000e+000],
[2.47412098e-014, 1.00000000e+000],
[6.41925512e-007, 9.99999358e-001],
[8.87706008e-015, 1.00000000e+000],
[1.00000000e+000, 7.27668630e-020],
[1.00000000e+000, 5.75617721e-022],
[1.00000000e+000, 4.83161832e-017],
[9.99571463e-001, 4.28537207e-004],
[1.00000000e+000, 2.76760752e-014],
[1.00000000e+000, 5.84136695e-100],
[1.00000000e+000, 4.19305275e-026],
[9.96580698e-019, 1.00000000e+000],
[7.84186796e-020, 1.00000000e+000],
[1.00000000e+000, 1.62440933e-055],
[5.11529722e-017, 1.00000000e+000],
[6.10023118e-010, 9.99999999e-001],
[3.53918511e-010, 1.00000000e+000],
[3.89701907e-014, 1.00000000e+000],
[2.76850277e-010, 1.00000000e+000],
[9.99999984e-001, 1.58812530e-008],
[1.00000000e+000, 6.62179158e-015],
[6.07278434e-009, 9.99999994e-001],
[1.86299027e-013, 1.00000000e+000],
[2.57800545e-015, 1.00000000e+000],
[8.67048632e-009, 9.99999991e-001],
[1.00000000e+000, 8.20268595e-021],
[3.06330200e-001, 6.93669800e-001],
```

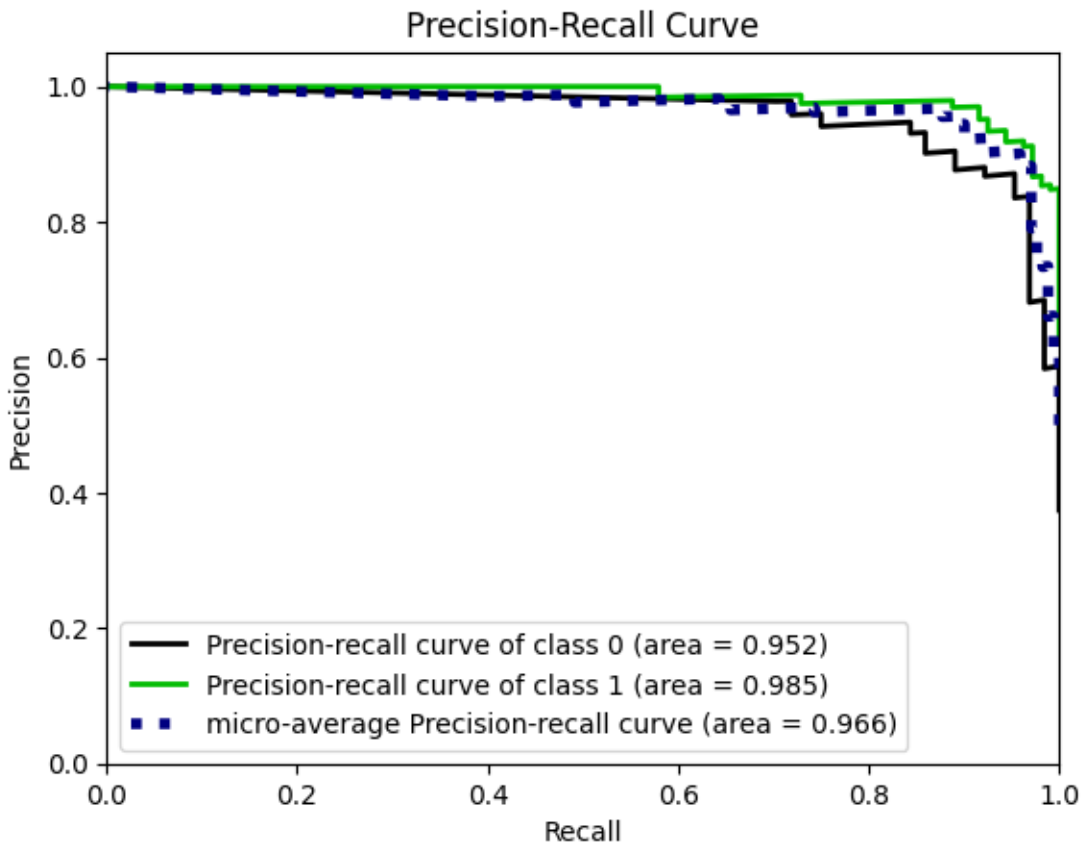
```
[2.76904635e-012, 1.00000000e+000],
[1.00000000e+000, 1.07090316e-030],
[1.00000000e+000, 5.88202001e-025],
[1.00000000e+000, 1.12952567e-049],
[1.00000000e+000, 1.37101315e-090],
[1.00000000e+000, 5.06841370e-012],
[6.87390643e-004, 9.99312609e-001],
[2.32024148e-014, 1.00000000e+000],
[7.98351890e-007, 9.99999202e-001],
[2.45345747e-010, 1.00000000e+000],
[1.00000000e+000, 4.96544130e-047],
[1.00000000e+000, 2.73594696e-026],
[3.34701403e-008, 9.99999967e-001],
[1.00000000e+000, 2.08754217e-057],
[9.99999998e-001, 2.43932083e-009],
[9.64990534e-003, 9.90350095e-001],
[8.02821659e-014, 1.00000000e+000],
[2.26325644e-011, 1.00000000e+000],
[1.00000000e+000, 2.69374767e-011],
[7.90120703e-012, 1.00000000e+000],
[1.13204641e-002, 9.88679536e-001],
[4.93482495e-011, 1.00000000e+000],
[9.99999878e-001, 1.22307191e-007],
[4.50662369e-012, 1.00000000e+000],
[8.25753085e-011, 1.00000000e+000],
[3.07725427e-015, 1.00000000e+000],
[1.00000000e+000, 2.85640504e-044],
[2.51559739e-002, 9.74844026e-001],
[4.61383390e-011, 1.00000000e+000],
[1.82186521e-016, 1.00000000e+000],
[5.28308302e-006, 9.99994717e-001],
[1.65228830e-011, 1.00000000e+000],
[1.00000000e+000, 4.56498688e-022],
[1.23132066e-017, 1.00000000e+000],
[1.22800876e-015, 1.00000000e+000],
[3.56992997e-014, 1.00000000e+000],
[2.03488382e-013, 1.00000000e+000],
[2.81915044e-016, 1.00000000e+000],
[2.36902189e-019, 1.00000000e+000],
[9.99995090e-001, 4.91045622e-006],
[4.83312761e-013, 1.00000000e+000],
[2.31509954e-014, 1.00000000e+000],
[9.32648265e-014, 1.00000000e+000],
[7.72754547e-016, 1.00000000e+000],
[8.43325608e-015, 1.00000000e+000],
[1.25510491e-017, 1.00000000e+000],
[1.29458768e-012, 1.00000000e+000],
```

```
       [6.16172274e-016, 1.00000000e+000],
       [5.64369777e-012, 1.00000000e+000],
       [1.00000000e+000, 2.20873654e-061],
       [7.71743562e-014, 1.00000000e+000],
       [2.00824407e-011, 1.00000000e+000],
       [2.77403674e-001, 7.22596326e-001],
       [1.00000000e+000, 2.19534740e-044],
       [1.00000000e+000, 2.50277461e-043],
       [6.29379257e-018, 1.00000000e+000],
       [1.00000000e+000, 2.98256222e-110],
       [1.06730721e-019, 1.00000000e+000],
       [1.00000000e+000, 2.50446769e-112],
       [6.39069091e-013, 1.00000000e+000],
       [1.00000000e+000, 1.47862500e-054],
       [5.65173547e-018, 1.00000000e+000],
       [8.37303002e-016, 1.00000000e+000],
       [5.84731461e-015, 1.00000000e+000],
       [2.86749633e-015, 1.00000000e+000],
       [7.62722007e-016, 1.00000000e+000],
       [7.16277436e-015, 1.00000000e+000],
       [6.90868938e-014, 1.00000000e+000],
       [5.36775578e-006, 9.99994632e-001],
       [1.00000000e+000, 1.09686447e-082],
       [2.21035391e-015, 1.00000000e+000],
       [5.08074190e-004, 9.99491926e-001],
       [8.51190961e-018, 1.00000000e+000],
       [1.30925836e-016, 1.00000000e+000],
       [1.24317330e-009, 9.99999999e-001],
       [1.00000000e+000, 2.47391080e-111],
       [1.87794810e-016, 1.00000000e+000],
       [2.36997051e-011, 1.00000000e+000],
       [8.62804098e-014, 1.00000000e+000],
       [5.10489927e-003, 9.94895101e-001],
       [2.25348614e-017, 1.00000000e+000],
       [5.95648811e-011, 1.00000000e+000],
       [7.42836733e-003, 9.92571633e-001],
       [2.03031410e-011, 1.00000000e+000],
       [2.39826926e-012, 1.00000000e+000],
       [1.02708885e-022, 1.00000000e+000]])
```

[84]:
```
plot_roc(y_test,y_test_pred_proba)
plt.show()
```

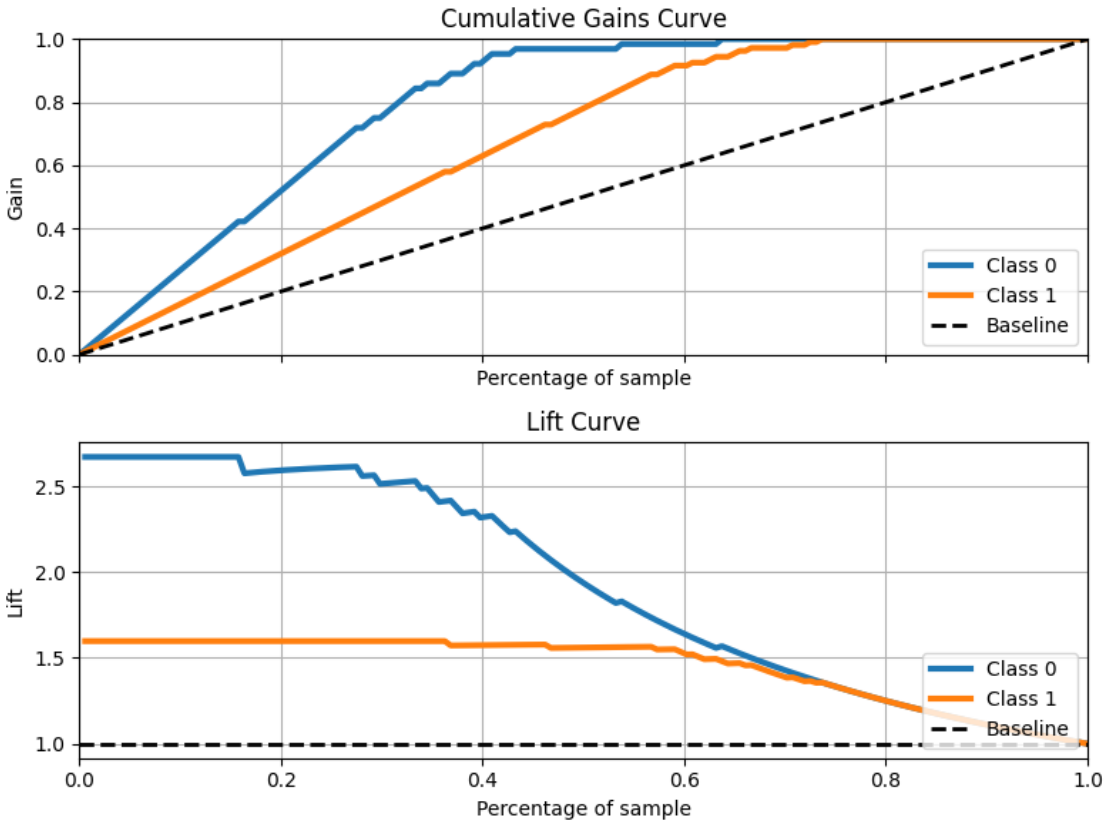


[85]:
```
plot_precision_recall(y_test, y_test_pred_proba)
plt.show()
```



For binary classification tasks you can also try to use cumulative gains curve and lift curve as implemented here

[86]:
```
fig, axs = plt.subplots(2, 1, sharex=True, figsize=(8,6))
plot_cumulative_gain(y_test, y_test_pred_proba, ax=axs[0])
plot_lift_curve(y_test, y_test_pred_proba, ax=axs[1])
plt.tight_layout()
plt.show()
```


Cumulative Gains Curve
Gain
Percentage of sample
Class 0
Class 1
Baseline
Lift Curve
Lift
Percentage of sample
Class 0
Class 1
Baseline

[86]:

[86]: